
CASE STUDY REPORT

API Testing for a Food Delivery Application

Vishwajit Mohol [92400120438]
Yagnik Bhojak [92400120585]

DEPARTMENT
COMPUTER ENGINEERING

ACADEMIC YEAR
2026-2027

SEMESTER
5TH

SUBJECT
OPEN SOURCE TECHNOLOGIES (01CE0526)

CLASS
5EV7

BATCH
A

CASE STUDY REPORT

RUBRICS FOR CASE STUDY

Sr. No.	Assessment Criteria	Excellent	Good	Satisfactory	Poor	Marks
1	Problem Analysis, Cloud Concepts & SDG Mapping	4 Marks: Clear problem identification, appropriate cloud concepts, and accurate SDG mapping.	3 Marks: Good analysis with minor gaps in concepts or SDG mapping.	1-2 Marks: Basic problem understanding with partial cloud concepts or SDG mapping.	0 Marks: Inadequate or incorrect analysis and no SDG mapping.	4
2	Security Analysis & Technical Depth	4 Marks: Comprehensive security analysis, well-defined architecture, appropriate technologies, and justified solutions.	3 Marks: Good technical explanation with minor omissions.	1-2 Marks: Limited security analysis and technical details.	0 Marks: No meaningful security analysis or technical explanation.	4
3	Report Quality & References	4 Marks: Well-structured report in IEEE format with comprehensive technical content, proper citations, and relevant references.	3 Marks: Good report structure with minor formatting or referencing issues.	1-2 Marks: Basic report with limited technical details or incomplete references.	0 Marks: Incomplete report with poor documentation and missing references.	4
Total						12

Sr. No.	Case Study Title / Problem Statement	COs	Date	R1(4)	R2(4)	R3(4)	Marks (12)	Signature
1	Food Delivery Application: API Testing and Error Handling Using Postman							

Ms. Priyanka Mangi
Subject Faculty

CASE STUDY REPORT

API Testing for a Food Delivery Application

Vishwajit Mohol
Dept. of Computer Engineering
Marwadi University, Rajkot, India
92400120438

Yagnik Bhojak
Dept. of Computer Engineering
Marwadi University, Rajkot, India
92400120585

Abstract—A food delivery application depends on reliable application programming interfaces (APIs) for operations such as creating orders, validating products, processing payments, and maintaining correct order states. Failures in these APIs can result in duplicate orders, invalid transactions, unauthorized access, and incorrect payment confirmations. This case study presents a systematic API testing approach for a food delivery application using Postman. The testing process focuses on valid and invalid order creation requests, duplicate-order handling, invalid product identification, quantity validation, authentication, payment processing, and repeated payment attempts. HTTP status codes and JSON response bodies are analyzed to determine whether the API behaves according to the expected functional and error-handling requirements. The study demonstrates how API testing can be used to identify functional defects and verify that invalid requests are handled through meaningful HTTP status codes such as 400 Bad Request, 401 Unauthorized, 404 Not Found, and 409 Conflict. The resulting test evidence provides a structured basis for evaluating API reliability, authentication enforcement, validation, and consistency of order and payment states.

Keywords— API Testing, REST API, Postman, HTTP, JSON, Authentication, Error Handling, Food Delivery Application, Software Testing

I. INTRODUCTION

A. Background of the Selected Case Study

The selected case study is based on a food delivery company whose customers have reported problems including login or authentication failures, duplicate orders, and incorrect payment confirmations. The objective is to design and perform API testing using Postman to identify functional defects and verify whether the API correctly handles valid, invalid, and repeated requests.

Modern food delivery applications rely heavily on backend APIs. A customer interacts with a mobile or web application, but operations such as creating an order, checking product availability, and processing payments are performed by backend services through APIs.

The API acts as an interface between the client application and the backend system. It receives an HTTP request, validates the supplied data, performs the required operation, updates the application state, and returns an HTTP response.

A defect in this layer can directly affect the application's behavior. For example, inadequate order validation may allow duplicate orders to be created. Incorrect input validation may allow invalid products or quantities to reach the application

logic. Similarly, weak authentication or incorrect payment-state validation may result in unauthorized or repeated payment operations.

B. Importance of API Testing

API testing is important because the API represents the communication contract between the client application and the backend system. A request being successfully transmitted does not necessarily mean that the underlying operation was performed correctly.

The HTTP status code, response body, validation behavior, authentication mechanism, and resulting application state must all be examined.

For a food delivery application, the API should accept valid orders, reject invalid products and quantities, prevent duplicate pending orders, protect payment operations through authentication, and prevent an order from being paid multiple times.

C. Objectives of the Case Study

The objectives of this case study are to:

- Design API test cases for the food delivery application.
- Test valid and invalid API requests.
- Test authentication for protected operations.
- Validate HTTP status codes and JSON responses.
- Verify duplicate-order prevention.
- Verify product and quantity validation.
- Verify successful payment processing.
- Verify that an already-paid order cannot be paid again.
- Analyze API error handling and identify possible improvements.

D. Scope of the Study

The study focuses on order creation, order validation, duplicate-order handling, authentication, payment confirmation, repeated payment attempts, HTTP response validation, and error handling.

The testing is performed against the locally running API using Postman.

CASE STUDY REPORT

II. APPLICATION BACKGROUND AND SCENARIO

A. Overview of the Application/System

The selected system represents a food delivery application in which customers can create orders for available products and subsequently make payments for those orders.

The application accepts structured JSON requests. An order contains information such as the customer's email address, product identifier, and requested quantity.

After successful order creation, the API returns information about the newly created order, including the order identifier, customer email, product identifier, quantity, total amount, and order status.

A successfully created order initially has a PENDING status. After successful payment, the order status changes to PAID.

B. Case Study Scenario

A food delivery company launches a mobile application. Customers report problems involving authentication, duplicate orders, and incorrect payment confirmations.

The problem is investigated at the API layer because order creation and payment operations are handled by backend endpoints.

The testing therefore focuses on determining whether:

- valid orders are successfully created;
- duplicate pending orders are rejected;
- invalid products are rejected;
- invalid quantities are rejected;
- protected payment operations require authentication;
- valid payments change the order state correctly; and
- already-paid orders cannot be paid again.

C. Testing Environment

The API was tested locally using Postman. The order creation endpoint used during testing was: POST /api/orders. The payment endpoint used during testing was: POST /api/orders/1/pay.

The API accepts JSON request data and returns JSON responses. The payment endpoint uses Bearer Token authentication.

III. PROBLEM ANALYSIS

A. Problem Identification

The main problems identified in the scenario are authentication failures, duplicate orders, and incorrect payment confirmations. These problems can occur when the backend does not properly validate incoming requests or does not correctly maintain the state of an order.

A duplicate order, for example, may be created if the API accepts the same order request repeatedly without checking whether an equivalent pending order already exists. Similarly, an incorrect payment confirmation may occur if the API processes a payment without checking whether the order has already been paid. Authentication problems may occur when protected operations do not properly verify the credentials supplied by the client.

B. Engineering Requirements

- The API testing strategy must verify both successful and unsuccessful execution paths.
- A valid request should produce the expected successful response and update the application state correctly.
- An invalid request should be rejected with an appropriate HTTP status code and a meaningful error message.
- The API must therefore validate input data, verify product existence, prevent duplicate pending orders, enforce authentication for protected operations, and prevent invalid payment state transitions.

C. Functional Requirements

- The API should create an order when valid customer, product, and quantity information is supplied.
- The API should reject requests containing a product that does not exist.
- The API should reject zero or otherwise invalid quantities.
- The API should prevent duplicate pending orders for the same product.
- The payment endpoint should require valid authentication.
- A valid authenticated payment request should change the order state from PENDING to PAID.
- A payment request for an already-paid order should be rejected.

D. Non-Functional Requirements

- The API should return consistent HTTP status codes and predictable JSON response structures.
- Error responses should clearly communicate the reason for failure.
- Protected operations should enforce authentication.
- Invalid input should be handled consistently.
- The API should prevent invalid state transitions.

CASE STUDY REPORT

IV. OPEN SOURCE SOLUTION ANALYSIS

A. Selected Open Source Technologies and Tools

- Node.js is used as the JavaScript runtime for the backend application.
- npm is used for dependency management and project execution.
- Postman is used to construct HTTP requests, send them to the API, inspect responses, and organize the API test cases.
- HTTP provides the communication mechanism between the API client and server.
- JSON is used as the primary format for request and response data.
- Git is used for version control, while GitHub is used to host the project repository and manage the source code.

B. Justification for Technology Selection

- Postman was selected as the primary API testing tool because the case study requires direct testing of REST API endpoints. It allows HTTP requests to be constructed with different parameters and request bodies, authentication credentials to be supplied, and the resulting status codes and JSON responses to be inspected.
- Node.js was used as the runtime environment for the backend API. It provides the environment required to execute the JavaScript-based server application and handle HTTP requests.
- npm was used for managing the Node.js project's dependencies and scripts. It provides a standard way to install and manage the packages required by the application.
- HTTP was used as the communication protocol between the Postman client and the backend API. The testing relies on HTTP methods and status codes to determine whether each request was processed successfully or rejected.
- JSON was used as the request and response data format. Its structured representation makes it suitable for transferring information such as customer details, product IDs, quantities, order states, and payment responses.
- Git and GitHub were used for version control and source-code management. They provide a way to maintain the project history and store the API implementation and related testing resources.

V. SYSTEM ARCHITECTURE AND TECHNOLOGY STACK

A. Overall Architecture

The system follows a client-server architecture. The client sends an HTTP request to the API. The server processes the request, performs validation and business logic, and returns an HTTP response containing a status code and JSON data.

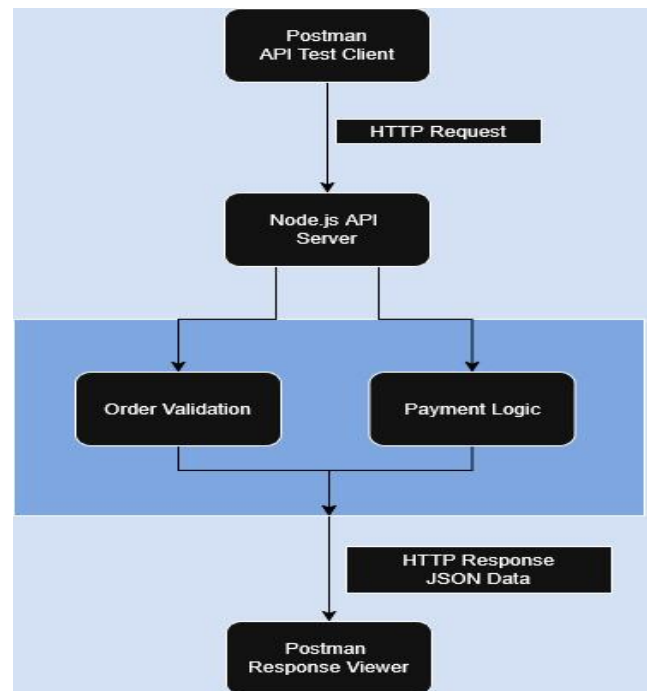


Fig. 1. Overall Architecture of the Food Delivery API Testing System

B. Technology Stack

Operation	HTTP Method	Endpoint
Create Order	POST	/api/orders
Pay Order	POST	/api/orders/1/pay

The order creation request accepts the fields customerEmail, productId, and quantity.

The payment endpoint requires Bearer Token authentication.

C. Request and Response Flow

For order creation, Postman sends a JSON request containing customer and product information. The API validates the request and either creates an order or returns an error.

For payment, Postman sends an authenticated request to the payment endpoint. The API verifies authentication and the current order state before confirming the payment.

CASE STUDY REPORT

VI. API TEST DESIGN AND IMPLEMENTATION

A. Create Valid Order

A valid order was created using a valid customer email, an existing product ID, and a positive quantity.

Expected Result: The API should create the order successfully and return 201 Created.

Actual Result: The API returned 201 Created. The response contained the order ID, customer email, product ID, quantity, total amount, and order status PENDING.

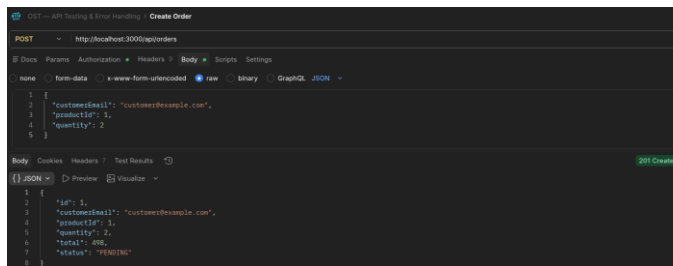


Fig. 2. Successful order creation response in Postman.

B. Duplicate Order

The same order request was submitted again for the same customer and product while the previous order was still pending.

Expected Result: The API should reject the duplicate pending order and return an appropriate conflict response.

Actual Result: The API returned 409 Conflict with the message Duplicate pending order for this product.

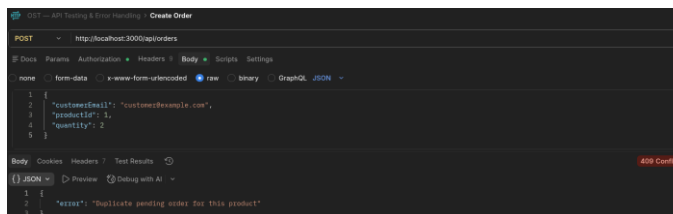


Fig. 3. Duplicate pending order rejected by the API.

C. Invalid Product

An order was submitted using a product ID that does not exist.

Expected Result: The API should reject the request because the specified product does not exist.

Actual Result: The API returned 404 Not Found with the message Product not found.

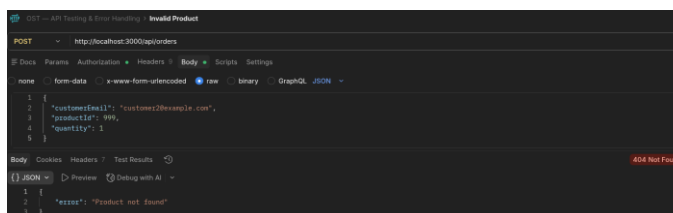


Fig. 4. Invalid product request rejected with 404 Not Found.

D. Invalid Quantity

An order was submitted with a quantity of zero.

Expected Result: The API should reject the request because the quantity must be a positive integer.

Actual Result: The API returned 400 Bad Request with the message quantity must be a positive integer.

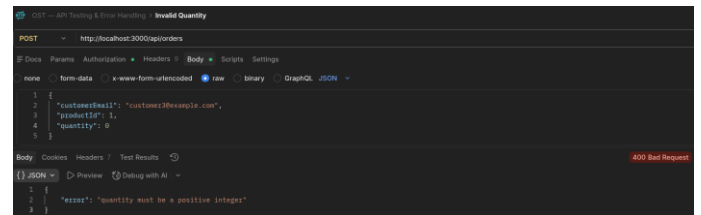


Fig. 5. Invalid quantity request rejected with 400 Bad Request.

E. Authenticated Payment

A payment request was submitted for a pending order using a valid Bearer Token.

Expected Result: The API should authenticate the request, process the payment, and change the order status from PENDING to PAID.

Actual Result: The API returned 200 OK with the message Payment confirmed. The response also showed the order status as PAID.

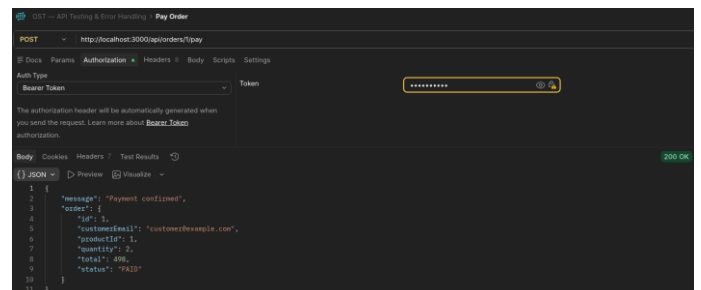


Fig. 6. Successful authenticated payment confirmation.

F. Order Creation Without Authentication

The order creation endpoint was accessed without providing valid authentication credentials.

Expected Result: The API should reject the request and return 401 Unauthorized.

Actual Result: The API returned 401 Unauthorized with the message Authentication required.

This confirms that authentication is enforced before an order can be created.

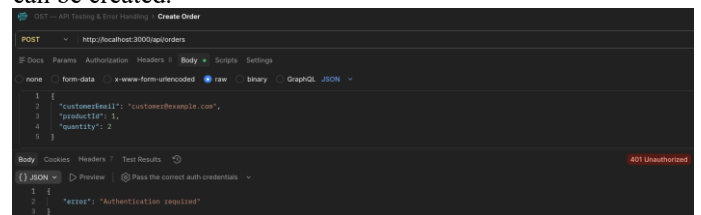


Fig. 7. Order creation request rejected due to missing authentication.

CASE STUDY REPORT

G. Payment Already Completed

After the order had already been successfully paid, the same payment request was submitted again.
Expected Result: The API should reject the payment because the order has already been paid.
Actual Result: The API returned 409 Conflict with the message: Order has already been paid.
This confirms that the API prevents repeated payment of the same order.

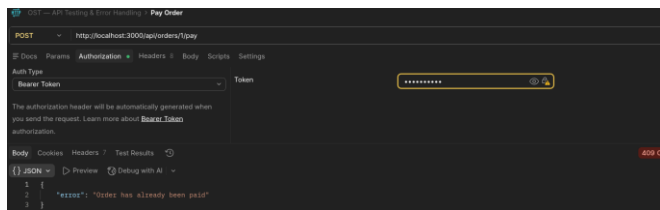


Fig. 8. Repeated payment request rejected for an already-paid order.

VII. TESTING RESULTS

Test Case	Expected Status	Actual Status	Result
Create Valid Order	201 Created	201 Created	PASS
Duplicate Order	409 Conflict	409 Conflict	PASS
Invalid Product	404 Not Found	404 Not Found	PASS
Invalid Quantity	400 Bad Request	400 Bad Request	PASS
Authenticated Payment	200 OK	200 OK	PASS
Order Creation Without Authentication	401 Unauthorized	401 Unauthorized	PASS
Payment Already Completed	409 Conflict	409 Conflict	PASS

All seven test scenarios produced the expected results.

The testing demonstrated that the API correctly handled both successful and unsuccessful requests within the tested scope.

VIII. TECHNICAL ANALYSIS

A. Working of the Proposed Solution

The API receives HTTP requests from the client and processes them according to the requested operation. For order creation, the API validates the authentication status, customer information, product ID, and quantity before creating an order. If the request is valid, a new order is created with a PENDING status and the API returns 201 Created.

For payment processing, the API first verifies the supplied Bearer Token and then checks the current state of the order. A valid payment request changes the order status from PENDING to PAID and returns 200 OK. If the order has already been paid, the API rejects the request with 409 Conflict.

Invalid requests are rejected before the operation is completed. For example, an invalid product produces 404 Not Found, an invalid quantity produces 400 Bad Request, and a request without valid authentication produces 401 Unauthorized.

B. Advantages

The proposed API testing approach verifies both successful operations and failure conditions. It confirms that invalid product IDs and quantities are rejected instead of creating invalid orders.

The testing also verifies that duplicate pending orders are prevented and that an already-paid order cannot be paid again. Authentication checks provide an additional layer of protection for API operations.

Using HTTP status codes and structured JSON responses also makes it easier for client applications to understand whether an operation succeeded or why it failed.

C. Limitations

The testing performed in this case study focuses on the available order and payment API operations.

A separate login endpoint was not tested directly. Authentication enforcement was tested by accessing the protected order creation endpoint without valid authentication credentials.

The tests also do not represent large-scale concurrent traffic. In a production environment, additional testing would be required to determine how the system behaves when multiple users submit orders or payment requests simultaneously.

The current testing also does not represent integration with an actual external payment gateway.

D. Scalability

The current implementation is suitable for demonstrating API functionality and error handling in a local environment. A production food delivery application would require persistent database storage and additional infrastructure to support a large number of simultaneous users and orders.

CASE STUDY REPORT

IX. SDG MAPPING AND JUSTIFICATION

A. Identify the Relevant Sustainable Development Goal

This case study maps most directly to SDG 9: Industry, Innovation and Infrastructure.

The case study focuses on testing the reliability and correctness of a software API that supports digital food delivery services. Reliable digital infrastructure is an important part of modern service platforms because applications depend on backend systems to process requests, maintain data, and perform transactions correctly.

B. Explain How the Proposed Open Source Solution Contributes Towards Achieving the Selected SDGs

The API testing solution contributes towards SDG 9 by improving the reliability and quality of the software infrastructure supporting the food delivery application.

The testing process verifies order creation, input validation, authentication, duplicate-order prevention, and payment-state validation. By identifying incorrect requests and verifying that the API returns appropriate responses, the testing process helps reduce failures that could affect customers and businesses.

The project also uses widely available development and testing technologies such as Node.js, Git, GitHub, JSON, HTTP, and Postman. This demonstrates how software systems can be developed, tested, and maintained using accessible development tools without requiring specialized proprietary infrastructure for the basic development and testing workflow.

C. Justification with Real-World Impact

The problems addressed in this case study have direct real-world consequences for digital food delivery services. A duplicate order can result in an unintended transaction, an incorrect payment state can create financial discrepancies, and inadequate authentication can allow unauthorized operations.

The testing approach demonstrated in this case study can be applied to production API development by validating both successful and failure conditions before software changes are released. The same principles can be extended through automated API testing, security testing, load testing, and CI/CD integration.

Therefore, the case study demonstrates how systematic API testing contributes to more reliable digital infrastructure, which directly relates to the objectives of SDG 9.

X. CHALLENGES AND RECOMMENDATIONS

A. Technical Challenges

One challenge in API testing is that a successful HTTP request does not automatically mean that the business operation is correct. The response status, response body, and application state must all be considered.

Another challenge is testing state-dependent operations. Payment behavior depends on the current state of an order, so the same request can produce different results depending on

whether the order is pending or already paid.

Authentication also requires separate testing because protected endpoints should behave differently when valid and invalid credentials are supplied.

B. Recommendations

The API should maintain consistent JSON structures for both successful and unsuccessful responses.

All input fields should continue to be validated before business logic is executed.

Quantity should be validated as a positive integer.

Payment endpoints should use proper authentication and authorization.

Payment operations should use idempotency mechanisms in a production environment to prevent accidental duplicate transactions.

Error messages should provide enough information for the client to understand the problem without exposing sensitive internal implementation details.

XI. CONCLUSION

A. Summary of the Case Study

API testing was designed and performed for a food delivery application using Postman. The testing covered order creation, duplicate-order prevention, product validation, quantity validation, authentication, payment confirmation, and repeated payment attempts. The API was evaluated using HTTP requests and JSON responses to verify both successful operations and error conditions.

B. Key Technical Findings

Seven test scenarios were executed successfully. Valid order creation returned 201 Created, authenticated payment returned 200 OK, invalid quantity returned 400 Bad Request, requests without valid authentication returned 401 Unauthorized, invalid product requests returned 404 Not Found, and duplicate or repeated payment operations returned 409 Conflict.

The testing demonstrated that the API correctly validates input, enforces authentication, prevents duplicate pending orders, and prevents payment of an order that has already been paid. These checks help maintain consistent application state and prevent common functional errors in order and payment processing.

C. Learning Outcomes

The study demonstrated that API testing is not limited to checking whether a request returns a successful response. Reliable testing requires validation of input data, authentication, HTTP status codes, response messages, and changes in application state.

The case study also demonstrated the importance of testing failure conditions. Testing invalid products, invalid quantities, duplicate orders, missing authentication, and repeated payments provided evidence that the API handles abnormal requests in a controlled manner.

CASE STUDY REPORT

D. Industrial Relevance of Open Source Technologies

The technologies used in this case study, including Node.js, npm, Git, GitHub, JSON, HTTP, and Postman, represent tools and technologies commonly encountered in modern software development and API workflows.

The testing approach demonstrated here can be extended to production systems through automated API test execution, CI/CD integration, security testing, load testing, monitoring, and database validation. This makes the case study relevant to real-world backend development, where reliable APIs are essential for maintaining dependable digital services.

REFERENCES

- [1] Postman, "Postman Documentation," [Online]. Available: <https://learning.postman.com/docs/>
- [2] Node.js, "Node.js Documentation," [Online]. Available: <https://nodejs.org/docs/latest/api/>
- [3] npm, "npm Documentation," [Online]. Available: <https://docs.npmjs.com/>
- [4] OWASP Foundation, "OWASP API Security Top 10," [Online]. Available: <https://owasp.org/projects/api-security-project>
- [5] Medium, "Getting Started with Postman for API Security" [Online]. Available: <https://medium.com/@nanaaisha1030/getting-started-with-postman-for-api-security-testing-6f284b847734>
- [6] V. Mohol and Y. Bhojak, "OST API Testing," GitHub repository, 2026. [Online]. Available: <https://github.com/Vishwajit1610/ost-api-testing>